

Kasuga Residences — Apartment Reservation System

Kasuga Residences is a web-based apartment reservation system that lets visitors browse published units and register as tenants, lets logged-in tenants request stays and complete checkout (including payment or pay-later paths), and track reservations, while authorized administrators manage rooms, bookings, users, announcements, and reviews. The stack is **PHP**, **MySQL**, and **server-side sessions** (e.g. XAMPP).

Users

User	Description
Guest	Any visitor who can use public pages (home, room catalog, about, services, contact) without logging in.
Tenant	A registered user (role tenant) who can book rooms, use checkout and My reservations, edit profile, and submit reviews for eligible stays.
Administrator	A user with role admin who uses the /admin area to manage catalog, bookings, users, announcements, and reviews.
System	The PHP application plus MySQL: validation, CSRF checks, optional Cloudflare Turnstile on sensitive forms, session handling, and mail (e.g. password reset) when SMTP is configured.

Start of the process

The process begins when someone opens the site. **Guests** see public content and can register or log in. **Tenants** with an active session are treated as logged in and can open booking and account pages. **Administrators** who sign in with an admin account use the admin dashboard instead of the public tenant-only tools.

Step-by-step process

A. Tenant registration

1. The user opens the site and goes to **Register**.
2. The user enters full name, email, phone, and password (and completes Turnstile if enabled).
3. The system validates input (format, strength, CSRF, captcha).

- If validation fails → the system shows field or general errors and stops.
 - If the **email is already registered** → the system rejects registration and shows an error.
4. If validation passes, the system **hashes the password**, inserts a new row in **users** with role **tenant**, and sets a success flash message.
 5. The user is redirected to **Login**.

B. Tenant login

1. The user opens **Login** and enters email and password (and Turnstile if enabled).
2. The system verifies CSRF and captcha.
 - If captcha fails → login does not proceed.
3. The system looks up **users** by email and verifies the password.
 - If credentials are wrong → the system shows an error.
 - If credentials are correct → the system **regenerates the session id**, stores **user_id** in the session, sets a success flash, and redirects (commonly to the home page).

C. Guest browsing (no login)

1. A **guest** opens **Home, Rooms, About, Services, or Contact** as implemented on the site.
2. The **home** page can show **active announcements** intended for the public (from **announcements**).
3. **Room listings** read **rooms** (and gallery data). Only **active** rooms appear where the code filters. Ratings may be aggregated from **reviews**.

D. Booking and checkout (tenant)

1. The **tenant** logs in and opens the room catalog, then chooses **Book** on an eligible room (**book.php** with **room_id**).
2. The system requires login; if the room is missing, inactive, or not open for booking → the system flashes an error and returns to the catalog.
3. The tenant enters stay details (check-in/out with times, guest count, notes) and a **payment choice** (e.g. pay now, pay later, pay at cashier).
4. The system validates the request and checks **date overlap** against existing **bookings** for that room in **pending** or **confirmed** status.
 - If the schedule conflicts → the system shows an error.
5. If the tenant already has a **pending** booking for the same room and dates, the system may reuse that booking for checkout or send the user to **My reservations**, depending on payment choice and whether payment was already submitted.
6. Otherwise the system may store a **checkout draft in the session** and redirect to **checkout.php**, or attach a booking id for continuing payment.
7. On **checkout.php**, the tenant reviews totals and completes the flow: confirm or abandon draft, attach payment proof / receipt fields / cashier or pay-later paths as implemented.
8. The system writes or updates **bookings** (amounts, status, payment timestamps, receipt reference, etc.) and redirects with flash messages, often to **My reservations**.

E. My reservations, profile, and reviews

1. The tenant opens **My reservations (my_bookings.php)** to see their **bookings** and statuses.
2. The tenant opens **Profile** to update account fields stored in **users**.
3. To **review** a stay (**review.php** with booking id), the system only allows bookings in **confirmed** or **checked_out** status for that user.
 - If allowed, the tenant submits a **1-5** rating and optional **comment** (length rules enforced); the system inserts **reviews** and returns to **My reservations**.

F. Administrator — rooms and catalog

1. An **admin** logs in (same login page; account must have **role = admin**).
2. The admin opens **/admin (admin.php** redirects to **admin/index.php**); **bootstrap.php** calls **require_admin()** on protected admin pages.
3. From **Rooms** (and related editors), the admin creates/edits **rooms** (code, title, description, rate, capacity, images, active flag, occupancy where used).

4. **Gallery** images are tied to **room_images** / configured URLs per the admin tools.

G. Administrator — bookings and notifications

1. The admin opens **Bookings** to list and manage reservations.
2. The admin can update **status**, payment fields, and operational fields (e.g. early checkout date) consistent with the admin UI.
3. **Notifications** (polling) surface events such as **new bookings** and **payment submitted**, using timestamps on **bookings** and optional “last seen” tracking for admins.

H. Administrator — users, announcements, reviews, archive, export

1. **Users** — the admin manages **users** (e.g. roles or basic account maintenance allowed by the pages).
2. **Announcements** — the admin maintains **announcements** (title, body, audience such as public/tenant/both, active flag, ordering).
3. **Reviews** — the admin can view or moderate per the admin reviews page.
4. **Archive / export** — the admin can use archive views and exported booking reports where implemented.

I. Password reset

1. The user opens **Forgot password** and submits identifying input (email flow per the app).
2. If a matching account exists, the system creates a token row in **password_reset_tokens** and sends an email with a reset link (SMTP when configured).
3. On **Reset password**, the user sets a new password; the system updates **users.password_hash** and clears or expires tokens appropriately.

J. Logout

1. The user opens **Logout**; the system clears and destroys the session, starts a clean session, sets a success flash, and redirects to the public home page.

System actions summary

Action	What the system does
Registration	Validates fields, CSRF, Turnstile; rejects duplicate email; hashes password; inserts users (tenant).
Login	Validates CSRF/Turnstile; verifies password; sets user_id in session; regenerates session id.
Browse rooms	Reads rooms (+ gallery); may show ratings from reviews.
Book	Requires login; checks room availability rules; prevents bad overlaps; session draft or pending booking handoff to checkout.php.
Checkout	Finalizes bookings row(s), payment metadata, statuses; clears or reuses session draft.
My reservations	Lists the tenant's bookings.
Review	Only for confirmed / checked_out bookings; inserts reviews.

Action	What the system does
Admin operations	require_admin(); CRUD/manage rooms, bookings, users, announcements, reviews, export/archive as built.
Password reset	Token in password_reset_tokens; email link; update users password.
Logout	Destroys session; redirect home.

End of the process

The process effectively ends when:

- A **tenant's stay** reaches a terminal state the app uses (confirmed, checked out, cancelled, etc.) and any payment/update steps the admin requires are done.
- A **tenant submits a review** for an eligible booking and returns to My reservations.
- An **administrator** finishes management tasks (bookings adjusted, rooms updated, announcements published) and **logs out**.
- A **guest** leaves after browsing or after registering / using contact without making a reservation.